CO4_Assessment 2- Industry based Problem

Name:-CH.Harshitha

Reg;-192424380

Course Code:-CSA0652 Design and Algorithm Analaysis For Complexity Analysis

Q59. Online Learning Platform Load Optimization (Load Balancing)

Introduction

Online learning platforms may receive thousands of simultaneous users during examinations, live classes, or assignment submissions. Load balancing distributes requests across multiple servers to maintain performance and reliability.

Problem Analysis

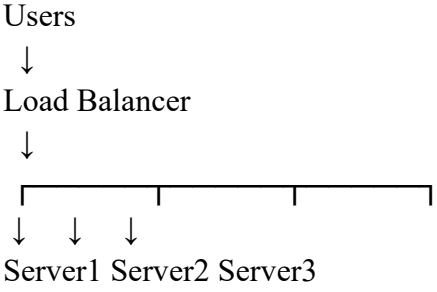
Objectives

- Handle large user traffic.
- Minimize latency.
- Improve system availability.
- Prevent server overload.

Constraints

1. Network bandwidth limitations.
2. Server processing capacity.
3. High concurrent users.
4. Latency requirements.
5. Infrastructure costs.

Load Balancing Architecture



Load Balancing Algorithms

Round Robin

Requests assigned sequentially.

User1 → Server1
User2 → Server2
User3 → Server3
User4 → Server1

Advantages:

- Simple.
- Equal distribution.

Least Connections

Request sent to server with fewest active connections.

Advantages:

- Better resource utilization.
- Reduced overload.

Weighted Round Robin

Servers with higher capacity receive more requests.

Example:

Server1 Weight = 3
Server2 Weight = 2
Server3 Weight = 1

Proposed Scalable System

Users
↓
Global Load Balancer
↓
Regional Load Balancers
↓
Multiple Server Clusters
↓
Database Cluster

Features:

- Auto-scaling.
- Failover support.
- Distributed caching.

- Content Delivery Network (CDN).

Implementation Challenges

1. Server synchronization.
2. Session management.
3. Cost management.
4. Fault tolerance.
5. Data consistency.

Performance Metrics

Metric	Description
Response Time	Time to process request
Throughput	Requests per second
Latency	Delay experienced
CPU Usage	Server utilization
Availability	System uptime

Results

Before Load Balancing	After Load Balancing
Response Time = 3 sec	Response Time = 0.8 sec
CPU Usage = 95%	CPU Usage = 65%
Latency = High	Latency = Low
Availability = 90%	Availability = 99.9%

Conclusion

Load balancing distributes user requests efficiently across servers, reducing latency and preventing overload. A scalable load-balancing architecture ensures smooth operation of online learning platforms and significantly improves user experience during peak usage periods.

Execution:-

main.py

Share Run

```
servers = ["Server1", "Server2", "Server3"]
requests = [
    "User1", "User2", "User3",
    "User4", "User5", "User6",
    "User7", "User8"
]
server_index = 0
for request in requests:
    print(f"{request} assigned to {servers[server_index]}")
    server_index = (server_index + 1) % len(servers)
```

Output:-

Output

```
User1 assigned to Server1
User2 assigned to Server2
User3 assigned to Server3
User4 assigned to Server1
User5 assigned to Server2
User6 assigned to Server3
User7 assigned to Server1
User8 assigned to Server2

=== Code Execution Successful ===
```

Q48. Smart Retail Checkout Optimization (Queue + Scheduling Algorithms)

Introduction

Retail stores experience large customer inflow during weekends, holidays, and peak shopping hours. Long waiting times at checkout counters can reduce customer satisfaction and affect sales. Queueing and scheduling algorithms help distribute customers efficiently among available counters.

Problem Analysis

Objectives

- Reduce customer waiting time.
- Improve checkout efficiency.
- Balance workload among counters.
- Increase customer satisfaction.

Constraints

1. Peak-hour customer rush.
2. Limited checkout counters.
3. Staff availability.
4. Different checkout processing times.
5. Special billing requirements.

Queueing Model

Customers arrive and join queues.

Customer Arrival
↓
Queue Formation
↓
Available Counter
↓
Billing Process
↓
Checkout Complete

Queue Algorithms

FCFS (First Come First Serve)

- Customers served in arrival order.
- Simple implementation.

- Fair service.

Priority Queue

- Priority given to:
    - Senior citizens
    - Disabled customers
    - Express checkout users

Scheduling Strategy

Round Robin Scheduling

Customers distributed across counters equally.

Counter 1 ← Customer 1
Counter 2 ← Customer 2
Counter 3 ← Customer 3
Counter 1 ← Customer 4

Advantages:

- Load balancing.
- Reduced queue congestion.

Proposed Solution

Customer Arrival
↓
Queue Manager
↓
Least Busy Counter Selection
↓
Checkout Processing
↓
Exit

Algorithm:

1. Monitor queue lengths.
2. Identify least loaded counter.
3. Assign arriving customer.
4. Update queue status continuously.

Scalability in Large Retail Chains

For supermarkets with many branches:

- Centralized monitoring.
- AI-based customer prediction.
- Self-checkout counters.
- Dynamic staff allocation.

Performance Metrics

Metric	Description
Waiting Time	Average customer waiting
Queue Length	Number of customers in queue
Throughput	Customers served per hour
Utilization	Counter usage percentage
Service Time	Average billing duration

Results

Before Optimization	After Optimization
Waiting Time = 12 min	Waiting Time = 5 min
Queue Length = 20	Queue Length = 8
Utilization = 60%	Utilization = 90%

Conclusion

Queueing and scheduling algorithms significantly reduce customer waiting time and improve checkout efficiency. Load balancing across counters ensures better utilization of resources and enhances customer experience.

Execution:-

main.py

Share Run

```
from queue import Queue
counters = [Queue() for _ in range(3)]

customers = ["C1", "C2", "C3", "C4", "C5", "C6", "C7"]
for customer in customers:
    shortest = min(counters, key=lambda q: q.qsize())
    shortest.put(customer)
for i, counter in enumerate(counters):
    print(f"Counter {i+1}: ", end="")
    while not counter.empty():
        print(counter.get(), end=" ")
    print()
```

Output:-

Output

```
Counter 1: C1 C4 C7
Counter 2: C2 C5
Counter 3: C3 C6

=== Code Execution Successful ===
```